

DynaMoE: Runtime Prefetch Horizon Control for Memory-Constrained Mixture-of-Experts Serving

Anonymous Author(s)

Abstract

A central bottleneck in serving Mixture-of-Experts (MoE) models on memory-constrained accelerators is the temporal mismatch between computation progress and expert parameter availability. Existing cross-layer prefetching systems hide this mismatch by forecasting expert usage and prefetching a fixed number of MoE layers ahead, but this fixed horizon S either leaves transfers exposed on the critical path (when S is too small) or triggers cache pollution and eviction cascades (when S is too large), and we show that no single offline S dominates across hardware, batches, and intra-batch token diversity. To address this, we propose treating S as a *runtime control variable* rather than a configuration knob, picking a different S_t at each control period in response to current stall and eviction signals. This reframing makes the prefetch depth track the per-period optimum, recovering the wins of any fixed S on the configurations where it would be best, simultaneously, and without offline tuning. To realize this idea, we develop DynaMoE, a serving runtime whose controller updates S via a bounded ± 1 rule with hysteresis on EMA-smoothed signals, at $O(1)$ per-update cost. A CPU-side delta-adjusted random-forest forecaster supplies confidence-weighted expert predictions without consuming GPU cycles. A residency-aware two-tier expert cache and a cache-aware token router bound the cost of misprediction so that every prediction error costs at most one DMA, never a cascade of stalls. We evaluate DynaMoE on three accelerators with very different compute-to-bandwidth ratios (NVIDIA A6000, NVIDIA H20, Ascend 910B) and three MoE models (DeepSeek-V2-Lite, Qwen1.5-MoE, Qwen2-MoE) under a 20 GB HBM cap. Compared to a fixed-horizon proactive-prefetching baseline, DynaMoE raises expert-activation prediction accuracy by an average of 21.8% (up to 30.4%) and reduces the cache-miss latency component by a geometric mean of $\approx 13\times$ across the eight supported model-device pairs (median $7\times\text{--}30\times$), while never degrading configurations in which transfers are already well overlapped.

CCS Concepts

• **Computer systems organization** $\rightarrow$ **Heterogeneous (hybrid) systems**; • **Computing methodologies** $\rightarrow$ *Massively parallel and high-performance simulations; Neural networks.*

Keywords

Mixture-of-Experts, LLM serving, expert prefetching, GPU memory management, adaptive control, parallel inference

1 Introduction

Mixture-of-Experts (MoE) architectures [14, 17, 28] let large language models (LLMs) scale parameter capacity without proportionally scaling per-token compute by activating only a small subset of experts per token [22]. Designs like Switch Transformer [11],

GShard [15], and DeepSeek-V2 [8] have made this routine. In latency-sensitive serving, however, MoE inference exposes a distinct systems bottleneck: expert weights are large, sparsely accessed, and rarely fit fully on the device, so *when* and *which* experts to move are as critical as routing itself.

The dominant mitigation today is *cross-layer prefetching*: while executing layer ℓ , the runtime forecasts the experts needed by the next S MoE layers and overlaps their transfer with computation. ProMoE [23], ExpertFlow [12], and Hobbit [25] all instantiate this template; they choose the prefetch depth S *offline* and reuse it across all requests. The depth itself, however, is what governs whether transfers ever land off the critical path. A too-small S leaves transfers unfinished when computation reaches layer ℓ , and the critical path accumulates *exposed waiting latency*. A too-large S makes speculative residency exceed HBM capacity, evictions cascade, and *cache pollution* amplifies subsequent miss latency. We measured this U-shaped tradeoff on three MoE models across three accelerators (Fig. 2). At a fixed model and device, picking a sub-optimal S inflates end-to-end latency by up to $2.5\times$ as batch size changes and by up to 300% as intra-batch token diversity changes (§ 2). Existing fixed-horizon systems pay these inflations in full whenever the deployment differs from their tuning point.

The position of the U-shape minimum shifts with effective transfer bandwidth (a hardware property), with batch-induced contention (a workload property), and with intra-batch routing diversity (a per-request property). For any candidate offline S^* , we can exhibit a workload on which some other $S' \neq S^*$ is strictly better (§ 2). Tuning S offline therefore amounts to picking the configuration on which to lose: the engineer chooses one device-workload pair on which to be optimal and accepts a multiplicative latency tax everywhere else. The right object is not a hyperparameter but a runtime control policy. A system that picks S_t at each control period in response to current stall and eviction signals can recover the wins of any fixed S *simultaneously*—on the configuration where it would be best, and without offline tuning—because the controller is free to choose a different S for every (device, batch, request, layer) tuple it encounters.

We present DynaMoE, a serving runtime that turns the prefetch horizon S into a first-class control variable. DynaMoE has three coordinated layers (Fig. 1(d)). A closed-loop controller updates S via a bounded ± 1 rule with hysteresis on EMA-smoothed stall and eviction signals at $O(1)$ per-update cost (§ 3.2). A CPU-side delta-adjusted forecaster refines pregate logits with token-, layer-, and horizon-aware features using a random forest that never touches the GPU (§ 3.3). A residency-aware two-tier expert cache with cache-aware token routing bounds the cost of misprediction so that every prediction error costs at most one DMA, never a cascading stall (§ 3.4).

We evaluate DynaMoE on NVIDIA A6000, NVIDIA H20, and Ascend 910B with DeepSeek-V2-Lite, Qwen1.5-MoE, and Qwen2-MoE

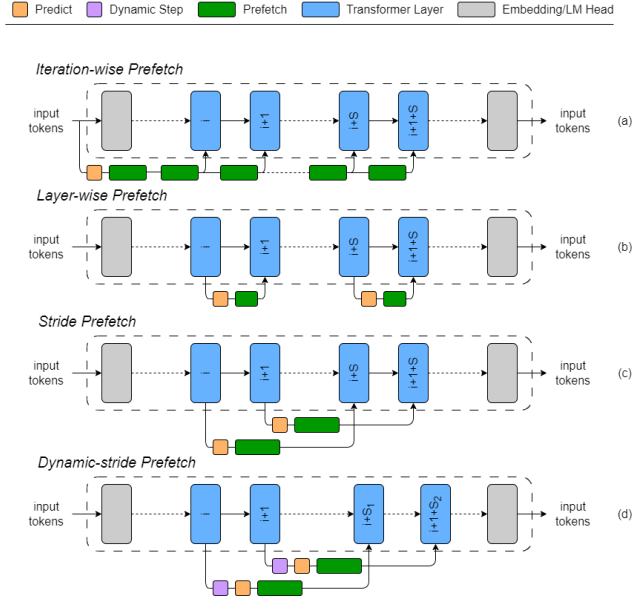


Figure 1: Cross-layer expert prefetching strategies. (a) Static expert–layer assignment, (b) reactive routing, (c) periodic prediction with a fixed horizon S (the template followed by ProMoE, ExpertFlow, and Hobbit), and (d) DynaMoE’s adaptive horizon control with feedback from runtime stall and eviction signals. (a)–(c) are dominated by some fixed S on every device–workload pair; (d) tracks the per-period optimum.

under a 20 GB HBM cap that forces expert swapping. Compared to a fixed-horizon proactive-prefetching baseline modeled after ProMoE [23] and Eliseev *et al.* [9], DynaMoE raises expert-activation prediction accuracy by an average of 21.8% (up to 30.4%) and reduces the cache-miss latency component by a geometric mean of $\approx 13\times$ across the eight supported model–device pairs (median $7\times$ – $30\times$), while never degrading configurations in which the baseline already overlaps most transfers (e.g., Qwen2-MoE on H20). The controller’s per-update cost stays under $50\ \mu\text{s}$ on a commodity CPU.

We summarize our contributions as follows.

- We characterize the U-shaped overlap–pollution tradeoff of cross-layer prefetching on three MoE models and three accelerators, and show that for any candidate offline S^* , the optimum lies elsewhere on some realistic workload—reframing S as a runtime control variable rather than a hyperparameter (§ 2).
- We design a stable single-variable controller: a bounded ± 1 update with hysteresis on EMA-smoothed signals tracks the per-period optimum at $O(1)$ per-update cost and is non-oscillatory by construction inside a configurable hysteresis band (§ 3.2, Algorithm 1).
- We co-design the forecaster and memory layer so that prediction errors are absorbed locally: a CPU-side delta-adjusted forecaster, a two-tier residency-aware cache, and cache-aware token routing together turn every prediction error into at most one DMA.

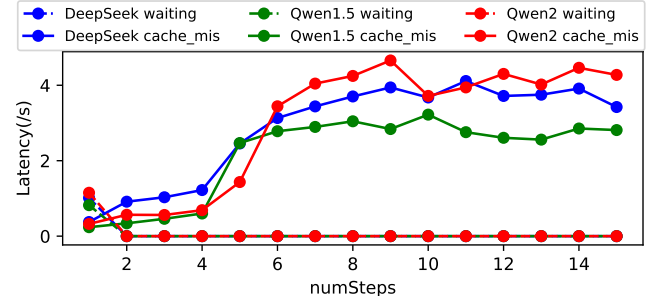


Figure 2: Latency vs. prefetch horizon S for three MoE models on NVIDIA A6000 under a 20 GB HBM cap. The y-axis is waiting + cache-miss latency (s/request); the x-axis is S (number of MoE layers prefetched ahead). Each curve is U-shaped, and the U-minimum sits at a different S for each model. We show in § 2.2 that the minimum also moves across devices, batches, and intra-batch token compositions, so no offline-tuned S is robust.

We isolate the contribution of each layer via ablations on three accelerators (§ 5).

2 Background and Motivation

2.1 MoE Serving and Cross-Layer Prefetching

A Mixture-of-Experts (MoE) layer routes each token through only a small subset of its expert sub-networks [6, 10, 22, 31, 37]. At inference time, this sparsity reshapes the per-layer cost from being *compute-bound* to being *memory-bound*: per-token FLOPs drop with the activation ratio, but each MoE layer must fetch a different subset of expert weights, and an MoE model’s expert footprint typically exceeds the device HBM by an order of magnitude. A serving runtime therefore cannot keep all experts resident; it must *predict* which experts will be activated next and *move* them onto the device ahead of execution.

Cross-layer prefetching is the dominant template for hiding these expert transfers. While computing layer ℓ , the runtime forecasts the experts likely to be activated in the next S MoE layers and issues their transfers on a separate stream so that they overlap with the compute of layers $\ell, \dots, \ell+S-1$. We call S the *prefetch horizon*. ProMoE [23], ExpertFlow [12], and Hobbit [25] all instantiate this template; they choose a single value of S at deployment time and reuse it across all requests. Crucially, S does not change the model’s routing semantics—it controls only how far ahead the system speculates.

The choice of S is what determines whether expert transfers stay off the critical path. Fig. 2 plots latency against S for three MoE models on NVIDIA A6000 under a 20 GB HBM cap. Each curve is U-shaped: below the U-minimum, transfers do not finish within the available compute window and the critical path accumulates exposed waiting; above it, speculative residency exceeds HBM capacity, evictions cascade, and cache pollution amplifies subsequent miss latency. The U-minimum sits at a different S for each model on the same device. Two questions follow: where does the U-minimum sit on a given deployment, and is it stable enough that we can find it offline?

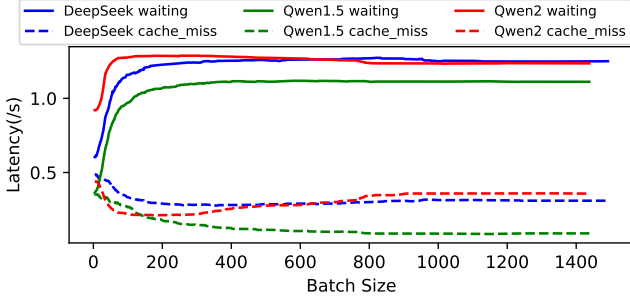


Figure 3: Latency components vs. batch size at a fixed S on A6000. The waiting component grows non-monotonically with batch size (more concurrent requests $\Rightarrow$ more concurrently demanded experts $\Rightarrow$ less per-expert overlap window), increasing by up to 2.5 $\times$, while the cache-miss component can decrease as residency stabilizes. The two components move in opposite directions, which is precisely why a single offline S cannot be optimal across batch sizes.

2.2 The Optimum Moves Along Three Independent Axes

The U-minimum is not stable. Three factors, each independent of the others, shift it. We discuss each in turn and connect it to the failure mode of fixed-horizon systems.

Hardware bandwidth. The accelerators used in this work span an order of magnitude in effective host-device bandwidth, from ~ 8 GB/s on AMD RX 6500XT to ~ 128 GB/s on NVIDIA H20 and Ascend 910B. The transfer time of a layer’s experts scales inversely with this bandwidth, while the per-layer compute time scales with FLOPs throughput; their ratio determines how many compute-layers’ worth of overlap window is available to hide one layer’s transfer. On a fast interconnect a small S already fits transfers inside the overlap window; on a slow one a much larger S is needed. A fixed-horizon system tuned for one accelerator therefore exposes systematic stalls when migrated to a slower interconnect or a co-tenanted GPU where effective bandwidth has dropped—this is precisely the cross-device brittleness of ProMoE-style runtimes.

Batch scale. Even on fixed hardware, the optimal S shifts with batch size. Larger batches amortize per-layer overhead but also widen the set of concurrently demanded experts and shrink each expert’s per-token overlap budget. To make this concrete, let E_{req} be the experts requested by a batch over the next S layers, $\hat{E}$ the experts actually prefetched, and $E_{\text{hit}} = E_{\text{req}} \cap \hat{E}$ those that arrive before they are needed. The expert miss ratio is $R_{\text{miss}} = 1 - \frac{|E_{\text{hit}}|}{|E_{\text{req}}|}$. As batch-induced demand diversity grows, $|E_{\text{req}}|$ grows faster than S can keep up, so R_{miss} rises. Fig. 3 shows that holding S fixed and sweeping batch size from 1 to 32 inflates the waiting component of latency by up to 2.5 $\times$ while the cache-miss component can simultaneously decrease as residency stabilizes. The two components move in opposite directions, so no single S minimizes both—a fixed-horizon system is forced to trade one for the other.

Intra-batch token diversity. At fixed batch size, requests with different token compositions activate different sets of experts in

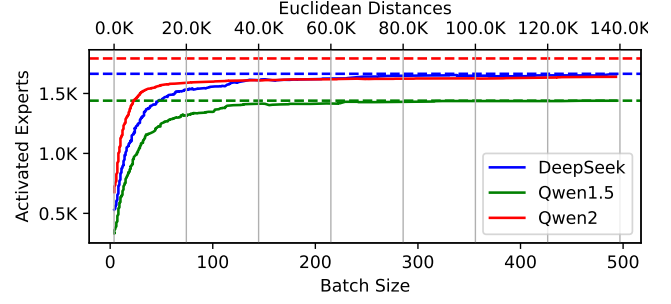


Figure 4: Number of distinct experts activated per layer vs. batch size (x-axis) and the embedding-dispersion proxy $\text{Dist}(t)$ (color), for DeepSeek-V2-Lite, Qwen1.5-MoE, and Qwen2-MoE. At fixed batch size, high-dispersion requests activate noticeably more experts and incur up to 300% latency variation, so dispersion is a useful per-request signal for the controller, on top of batch size.

subsequent layers. We use a lightweight proxy for intra-batch diversity, the sum of pairwise embedding distances:

$$\text{Dist}(t) = \sum_{1 \leq i < j \leq k} \|\mathbf{v}_{t_i} - \mathbf{v}_{t_j}\|_2,$$

where $\mathbf{v}_{t_i}$ is the embedding of token t_i . Fig. 4 plots the number of distinct experts activated per layer against batch size and $\text{Dist}(t)$ on three MoE models. At a fixed batch size, requests with high semantic dispersion activate noticeably more experts and exhibit up to 300% end-to-end latency variation compared to low-dispersion requests with the same batch size. A fixed-horizon system that chooses S on a representative low-dispersion batch therefore systematically under-prefetches on diverse batches, even when the deployment hardware and batch size are unchanged.

2.3 Implications for System Design

Putting the three axes together, the U-minimum drifts continuously across (device, workload, request) tuples. For any candidate offline S^* , we can construct a deployment in which some other $S' \neq S^*$ is strictly better, and we exhibit such cases in our evaluation (§ 5). Tuning S offline therefore amounts to picking the configuration on which to lose: the engineer chooses one device-workload pair to optimize and accepts a multiplicative latency tax everywhere else. The right object is not a hyperparameter but a control policy that chooses S_t at each control period in response to current runtime signals. Building such a policy means solving three coupled sub-problems—deriving a stable S_t from partial, noisy observations of bandwidth, demand, and cache state; predicting which experts will be needed over the next S_t layers without competing for GPU cycles; and bounding the cost of misprediction so that errors cause at most one DMA, never a cascade of stalls—which we address in § 3.2, § 3.3, and § 3.4, respectively.

3 System Design

This section answers the three coupled sub-problems raised in § 2.3, layer by layer: § 3.2 derives a stable S_t from partial observations; § 3.3 predicts the experts needed over the next S_t layers without

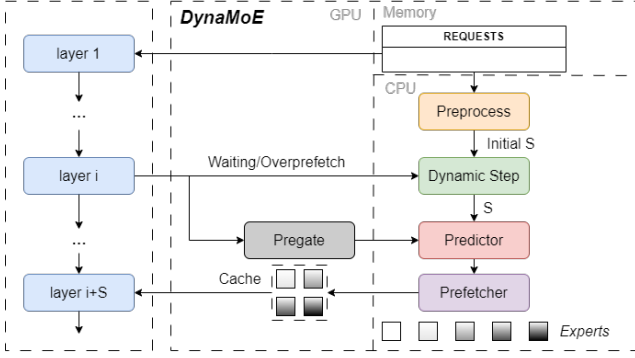


Figure 5: Architecture of DynaMoE. *Online path:* (1) per-layer hooks read smoothed waiting and miss signals ($\tilde{w}_t, \tilde{m}_t$); (2) the controller updates the horizon S_t (1); (3) the forecaster emits the expert set $\hat{E}_{t+1..t+S_t}$ and a confidence score; (4) the two-tier cache issues asynchronous prefetches for non-resident experts on a dedicated transfer stream; (5) the cache-aware router executes tokens whose experts are already resident first, deferring the rest until prefetch completes. *Offline path:* the forecaster’s CPU-side random forest is trained from logged (t, ℓ, S, a) tuples; no GPU cycles are spent on prediction at serving time.

consuming GPU cycles; and § 3.4 bounds the cost of misprediction so that errors cause at most one DMA, never a cascade of stalls. We start with the system overview that ties them together (§ 3.1).

3.1 Overview and Architecture

DynaMoE converts MoE inference into a closed-loop scheduling problem over a single scalar S_t . The system has three layers (Fig. 5). The *controller* (§ 3.2) updates S_t from EMA-smoothed runtime signals at constant per-update cost. The *forecaster* (§ 3.3) predicts the experts likely to be activated over the next S_t layers using token-, layer-, and horizon-aware features, on the CPU. The *memory layer* (§ 3.4) materializes the forecast through asynchronous transfers, a two-tier cache that protects high-reuse experts, and cache-aware token routing that lets cache misses be absorbed in parallel with resident-expert execution.

Each control period T_u , the controller solves the following optimization:

$$\begin{aligned} \min_{S_{t+1} \in \mathbb{Z} \cap [S_{\min}, S_{\max}]} & \tilde{L}_{\text{wait}}(S_{t+1}) + \lambda \tilde{L}_{\text{miss}}(S_{t+1}), \\ \text{s.t.} & \quad \text{(C1) } \text{occ}(\hat{E}_{t+1..t+S_{t+1}}) \leq M_{\text{HBM}}, \\ & \quad \text{(C2) } |S_{t+1} - S_t| \leq 1, \\ & \quad \text{(C3) } \text{prefetch} \subseteq \hat{E}_{t+1..t+S_{t+1}}. \end{aligned} \quad (1)$$

$\tilde{L}_{\text{wait}}$ and $\tilde{L}_{\text{miss}}$ are EMA-smoothed exposed-stall and eviction-pressure proxies (§ 3.2); $\hat{E}$ is the forecaster’s prediction (§ 3.3); M_{HBM} is the expert residency budget. (C1) keeps each candidate S realizable under the HBM cap, (C2) bounds per-step change so the controller cannot oscillate, and (C3) keeps speculative prefetching restricted to the current prediction set. DynaMoE exposes only two scalar hyperparameters—the regime weight λ and the hysteresis threshold θ_h used by the controller—both fixed once at startup.

The rest of this section answers, in order:

- **Q1.** How can the controller pick a stable S_t from partial, noisy observations of bandwidth, demand, and cache state? (§ 3.2)
- **Q2.** How can a predictor extend pregate signals over long horizons without competing for GPU cycles? (§ 3.3)
- **Q3.** How can the memory layer bound the cost of misprediction so that errors cause at most one DMA, never a cascade of stalls? (§ 3.4)

3.2 Adaptive Controller

The controller maintains EMA-smoothed estimates of two runtime signals: $\tilde{w}_t = \text{EMA}_\alpha(w_t)$ for exposed waiting (the fraction of MoE-layer time spent stalled on a missing expert) and $\tilde{m}_t = \text{EMA}_\alpha(m_t)$ for pollution pressure (evictions per miss-induced refill). Smoothing decouples the controller from per-layer noise without introducing learned state. The signed quantity $\Delta_t = \tilde{w}_t - \lambda \tilde{m}_t$ identifies the dominant failure mode at each control period: $\Delta_t > 0$ means waiting dominates (under-prefetch), $\Delta_t < 0$ means pollution dominates (over-prefetch); the weight λ trades off the two costs, and we use $\lambda = 1$ throughout this paper.

The controller updates S by at most one layer per period inside a hysteresis band of width θ_h : $S_{t+1} = \min(S_t + 1, S_{\max})$ if $\Delta_t > \theta_h$; $\max(S_t - 1, S_{\min})$ if $\Delta_t < -\theta_h$; and S_t otherwise. The rule satisfies constraint (C2) of Eq. 1 by construction. A short informal argument explains stability. Inside the band the rule is the identity, so once the regime indicator enters the band, S_t is fixed. Outside the band, $|\Delta_t| > \theta_h$ deterministically pushes S_t toward the regime knee at one layer per period; bounded by $[S_{\min}, S_{\max}]$, the trajectory reaches a band-boundary in at most $S_{\max} - S_{\min}$ periods. EMA smoothing ($\alpha \in [0, 1]$) exponentially attenuates the indicator’s response to any single noisy observation, so the controller cannot be flipped in and out of the band by a transient spike.

1 summarizes one control period. The per-update cost is $O(1)$; the aggregate per-request cost is $O(L_{\text{MoE}})$, independent of model size, batch size, and S . An update completes in under $50 \mu\text{s}$ on a commodity CPU in our implementation. Lines 1–2 update the smoothed signals; line 3 computes Δ_t ; lines 4–10 are the ± 1 rule above. Across one request the controller has three typical phases: in the first periods after a workload change Δ_t is consistently outside the band and S moves monotonically toward the new knee; around the knee Δ_t enters the band and S holds; under sustained drift (a co-tenant traffic spike, or a batch with very different intra-batch dispersion) Δ_t leaves the band again and the controller re-converges.

Before any feedback is available, S is initialized from a coarse analytical estimate using forecaster output and recent transfer timing. With N_e predicted experts per layer of size E_s bytes, measured effective bandwidth C_s , and per-layer compute time T_l , the transfer of one layer’s experts takes $N_e E_s / C_s$ seconds, which the controller hides behind $S \cdot T_l$ seconds of compute. Solving for S gives $S_0 = \lceil N_e E_s / (C_s T_l) \rceil$, in units of MoE layers. The closed-loop rule then takes over at the first control period; mis-set S_0 only delays convergence.

3.3 Forecasting Layer

For each request DynaMoE logs a tuple $(t_i, \ell_i, p_i, a_i, s_i)$ per MoE layer, recording token IDs, layer index, pregate-predicted experts,

Algorithm 1 DynaMoE adaptive horizon controller (per control period T_u).

Require: EMA factor α , regime weight λ , hysteresis θ_h , bounds $[S_{\min}, S_{\max}]$

Require: previous state $(\tilde{w}_{t-1}, \tilde{m}_{t-1}, S_t)$, raw counters w_t, m_t

Ensure: next horizon S_{t+1}

```

1:  $\tilde{w}_t \leftarrow \alpha \tilde{w}_{t-1} + (1 - \alpha) w_t$             $\triangleright$  smoothed waiting
2:  $\tilde{m}_t \leftarrow \alpha \tilde{m}_{t-1} + (1 - \alpha) m_t$         $\triangleright$  smoothed pollution
3:  $\Delta_t \leftarrow \tilde{w}_t - \lambda \tilde{m}_t$                   $\triangleright$  regime indicator
4: if  $\Delta_t > \theta_h$  then                          $\triangleright$  stall-dominated
5:    $S_{t+1} \leftarrow \min(S_t + 1, S_{\max})$ 
6: else if  $\Delta_t < -\theta_h$  then                    $\triangleright$  pollution-dominated
7:    $S_{t+1} \leftarrow \max(S_t - 1, S_{\min})$ 
8: else                                          $\triangleright$  within hysteresis band
9:    $S_{t+1} \leftarrow S_t$ 
10: end if
11: return  $S_{t+1}$ 

```

actually activated experts, and the step size in effect. Tokens are embedded through a frozen table $E \in \mathbb{R}^{V \times d}$ and mean-pooled into a request descriptor $e = \frac{1}{k} \sum_{i=1}^k E_{t_i}$; the per-layer feature $x = [e, S, \ell, \text{prev_act}]$ concatenates this descriptor with the current horizon, layer index, and a binary vector of prior-layer activations. A CPU-side random forest regresses x to the multi-hot label y of actually activated experts under MSE. We chose a forest over a deep predictor for three system-level reasons: it adds no GPU inference, its prediction latency at control-period granularity is sub-millisecond on commodity CPUs and never blocks the inference critical path, and its accuracy is stable across our model groups (§ 5).

Raw pregate outputs exhibit layer-local noise. The forecaster learns the deviation Δ between pregate predictions and actual activations and produces a refined prediction $\hat{y} = p + \Delta$. When an identical (token-sequence, layer, S) tuple reoccurs, the cached prediction is reused; otherwise the system falls back to the model's own top- k router logits, so DynaMoE never blocks waiting for prediction. The forecaster passes two signals to the controller: the expected number of experts per future layer N_e (used in S_0 in § 3.2), and a confidence score derived from historical prediction accuracy. Low confidence widens $\hat{E}$, which tightens (C1) of Eq. 1 and implicitly shrinks S , so the controller becomes conservative when the predictor is uncertain.

3.4 Expert Memory Management

The memory layer translates forecasts into residency decisions and returns cache statistics to the controller. It is designed so that a mispredicted expert costs at most one extra DMA, never a cascading stall, which is what lets the controller safely explore larger S without paying eviction-reload amplification.

Two-tier LRU. Rather than a single eviction queue, DynaMoE maintains two LRU structures: LRU_high holds experts with demonstrated reuse (recently accessed or imminently predicted), while LRU_low holds the rest. Evictions are drawn preferentially from LRU_low, which spills to host DRAM and preserves high-reuse experts on the device. This two-tier design directly addresses a

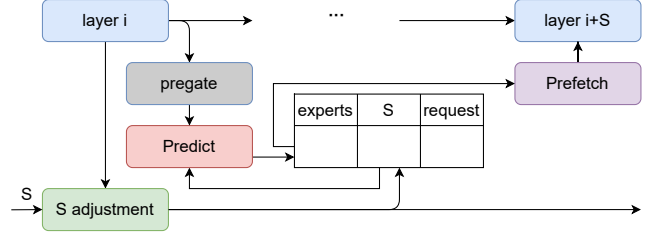


Figure 6: Co-design of the forecaster and the controller in DynaMoE. The forecaster provides $\hat{E}_{t+1..t+S}$ and a confidence score; the controller consumes the predicted expert count N_e to compute S_0 at startup, and the smoothed counters $\tilde{w}_t, \tilde{m}_t$ to update S at each period. Low-confidence forecasts widen $|\hat{E}|$, which tightens the HBM constraint (C1) and implicitly shrinks S , so the controller automatically becomes conservative when the predictor is uncertain.

pathology of single-tier LRU on layer-wise MoE: experts from the first S layers are reused later during decoding, yet a single LRU prematurely evicts them as more recent layers fill the queue.

Prefetch-cache feedback loop. Memory management and the controller form a closed loop: (1) before computing S , the controller queries current cache occupancy to estimate effective bandwidth and the anticipated eviction cost; (2) after each prefetch completes, observed transfer time updates the bandwidth estimate C_s used in S_0 (§ 3.2); (3) cache hit/miss counters feed $\tilde{m}_t$ in 1. This feedback prevents the runtime from issuing prefetches that would themselves trigger evictions of experts the controller still expects to use.

Cache-aware routing. A cache miss inside an MoE layer manifests as an asynchronous stall. DynaMoE exploits the asynchrony by routing tokens to experts in order of cache residency: tokens whose experts are already resident execute first; tokens that would trigger a fault are deferred until the corresponding swap-in completes, which happens in parallel with the resident-expert path. Compared with conventional sequential swap-in (where every miss blocks), this overlap reduces the exposed component of cache-miss latency without altering routing semantics.

4 Implementation

We implement DynaMoE as a drop-in runtime layer on top of HuggingFace Transformers [29], with ~4.5K lines of Python and ~0.6K lines of CUDA. The same Python codebase compiles for both NVIDIA (PyTorch 2.3 / CUDA 12.2) and Ascend (CANN 7.0): a thin ~150-LoC device shim hides the difference between CUDA streams and the Ascend runtime equivalents, so the controller and forecaster are device-independent. The implementation modifies neither the model architecture nor the routing/gating semantics, and can be swapped in front of any MoE model that follows HuggingFace's *MoeSparseMoeBlock convention.

The forecaster is trained from (t, ℓ, S, a) tuples logged in a fixed-size pinned ring buffer (default 64K entries) during serving. Retraining runs on the controller's CPU thread once per epoch (10K requests or 60 seconds, whichever comes first) and never touches the GPU. On a previously unseen model the buffer is empty for the first epoch and the forecaster falls back to the model's own pregate

top- k , so DynaMoE degrades to a baseline-equivalent state at cold start rather than failing.

Latency attribution. For each MoE layer the runtime emits four `torch.cuda.Event` timestamps—compute begin, compute end, first stall on a missing expert, and stall release—and aggregates them in a per-request ring buffer. The difference between stall release and stall begin is the *exposed waiting* component; the time spent inside the swap-in path is the *cache-miss* component; compute time is reported separately. The breakdown plotted in § 5, including Tables 1–3, is read directly from these counters. Seeds are fixed for ShareGPT bucketing and forecaster training; we will release the runtime, device shim, training pipeline, and benchmark harness at the camera-ready URL.

5 Evaluation

We evaluate DynaMoE on three accelerators with very different compute-to-bandwidth ratios and three MoE models under a 20 GB HBM cap. Throughout, we report two latency components measured by the instrumentation in § 4: *exposed waiting* (stall time not overlapped by compute) and *cache-miss latency* (time spent inside the swap-in path). Compute time is reported separately and excluded from both. We summarize the headline findings as follows:

- DynaMoE reduces the cache-miss latency component relative to a fixed-horizon baseline by a geometric mean of $\approx 13\times$ across the eight supported model-device pairs, while never degrading configurations in which the baseline already overlaps most transfers (§ 5.2).
- Each component contributes independently: the forecaster raises prediction accuracy by 21.8% on average (up to 30.4%, § 5.3); the two-tier cache prevents the eviction-reload cliff that fixed-LRU suffers near $S \approx 4$ (§ 5.4); cache-aware routing suppresses the residual cache-miss component on H20, where higher compute throughput exposes more transfer time (§ 5.5).
- Aggregate runtime overhead is below 50 μ s per controller update on a commodity CPU, and the forecaster’s sub-millisecond inference never blocks the GPU critical path (§ 5.6).

5.1 Experiment Setup

Testbeds. All experiments were conducted on three accelerator platforms: NVIDIA A6000 [18] (64 GB/s transfer bandwidth), NVIDIA H20 (128 GB/s), and Ascend 910B (128 GB/s). DynaMoE is implemented in Python and CUDA based on the Transformers framework [29]. To reduce memory fragmentation and enable fine-grained expert residency control, GPU HBM and host DRAM are managed in block granularity, with dynamic allocation driven by runtime demand. Expert swap-in/out transfers are issued on a dedicated CUDA stream to enable overlap with compute, while a separate CPU thread tracks transfer progress and memory block states. We enable continuous batching to emulate serving-style throughput-oriented execution.

Workloads and Models. Requests are sampled from ShareGPT [1]. To mitigate heavy-tailed length distribution, we bucket requests by prompt length and sample up to 50 requests per bucket (within

$\pm 5\%$ length variation). We evaluate three MoE models: DeepSeek-V2-Lite [3, 7, 8], Qwen1.5 [19, 26], and Qwen2.0 [20, 27]. We apply INT4 quantization to Qwen2.0 where supported.

Memory cap and comparability. To emulate resource-constrained serving, we cap GPU memory at 20 GB on all platforms. This cap forces expert residency competition and makes cache behavior observable. Ascend 910B currently lacks INT4 support in our software stack, which prevents running Qwen2.0 under the same 20 GB cap. We therefore report Qwen2.0 results on A6000 and H20, and report DeepSeek/Qwen1.5 on all three devices. This limitation is explicitly reflected in tables and does not affect our cross-device conclusions about the mechanisms of DynaMoE, which are validated on two MoE models across all platforms.

Baselines. We compare DynaMoE against a baseline implementation built on Transformers with representative proactive expert-loading components. Specifically, the baseline incorporates (i) periodic cross-layer prediction in the style of ProMoE [23], and (ii) pregate-based signals commonly used to approximate upcoming expert activations [9]. We use the term *Baseline* to denote this combined implementation. When focusing on prediction accuracy alone, we separately report pregate accuracy and compare against our Predictor (and discuss trend differences relative to periodic prediction).

5.2 Q1: Overall Waiting Latency Declines

We first evaluate the end-to-end impact of DynaMoE on exposed waiting and cache-miss latency under memory-capped inference. Conceptually, DynaMoE reduces waiting by (i) forecasting the experts that will be needed before execution reaches them, (ii) prefetching these experts early enough to overlap transfer with compute, and (iii) adapting the forecasting horizon (step size) to match the current compute-communication ratio and workload dynamics.

We construct a test suite with prompt lengths from 8 to 1024 and evaluate DeepSeek, Qwen1.5, and Qwen2.0 (where supported). Fig. 7 shows that DynaMoE reduces the combined waiting and cache-miss components relative to the baseline on every device-model pair. The improvement is consistent across the three heterogeneous platforms, indicating that DynaMoE adapts to device-specific overlap opportunities rather than relying on a single offline-tuned step size. The magnitude of reduction is largest where expert transfer is on the critical path (Qwen1.5 exhibits the largest gains), and smaller but still positive where the model already alleviates cache pressure architecturally (Qwen2.0, whose shared experts [30] partially absorb the problem). Both latency components shrink together, which means DynaMoE improves overlap without inducing cache thrashing.

A counter-intuitive cross-device data point reinforces the runtime-control argument. H20’s higher peak compute does not always translate into lower end-to-end latency under our 20 GB cap: at small initial S , H20 finishes per-layer compute quickly and then stalls on transfers because effective bandwidth (and memory-allocator overhead) does not scale with FLOPs. A6000’s slower compute hides a larger fraction of the same transfer naturally. This is exactly the brittleness our motivation predicts—the optimal S shifts with the

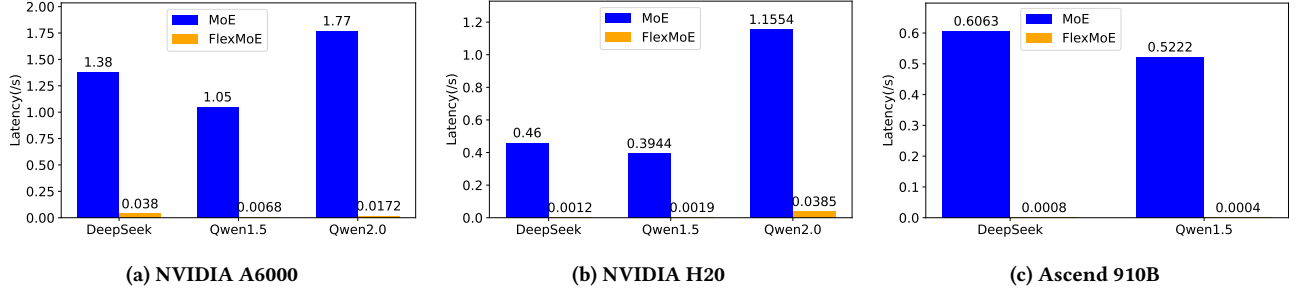


Figure 7: Overall waiting + cache-miss latency under memory-capped MoE inference (prompt length 8–1024).

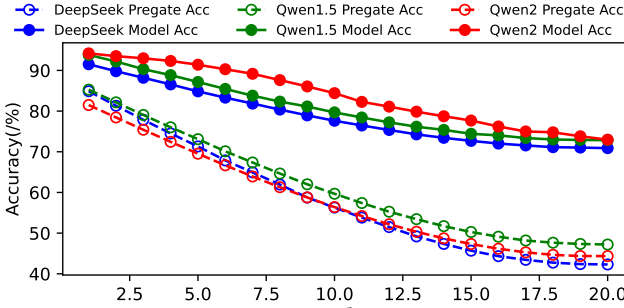


Figure 8: Expert activation prediction accuracy: pregate vs. Predictor.

compute-to-bandwidth ratio—and DynaMoE converges to different S_t on the two devices, recovering the device-specific overlap opportunity in both.

5.3 Q2: Pre-Gate-Based Predictor

We next isolate the contribution of the forecaster, since prediction quality directly affects how often experts are resident when needed and how much over-prefetch pollutes HBM. The forecaster operates on token-level features and activation history, so its accuracy is independent of GPU device; we therefore report it on A6000 as a representative platform, evaluating 410 requests of varying lengths under different step numbers when skipping layers in DeepSeek, Qwen1.5, and Qwen2.0.

Fig. 8 shows that the forecaster improves accuracy consistently across all three model groups: the average gain over pregate is 21.79% (up to 30.36%). It also degrades more smoothly as the prediction horizon grows, indicating better robustness to long-range forecasting than pregate-only signals; compared with ProMoE-style proactive prediction [23], the forecaster achieves higher accuracy at the same step size.

Fitting both accuracy curves to an exponential decay, the asymptotic accuracy gap Δ_∞ ranges from 30.8 to 37.0 percentage points across the three model groups, confirming that the forecaster’s advantage over pregate-only signals is stable at long horizons.

Under cross-layer prefetching, prediction errors incur two complementary costs: under-forecasting misses required experts and exposes waiting, while over-forecasting loads unused experts, consuming bandwidth and polluting HBM. Improving prediction accuracy therefore reduces both components, and Fig. 9 confirms this

Table 1: Cache-miss latency (s/request) under the fixed-horizon baseline.

Model	A6000	H20	Ascend 910B
DeepSeek-V2-Lite	1.38	0.46	0.61
Qwen1.5-MoE	1.05	0.39	0.52
Qwen2-MoE	1.77	1.16	–

Table 2: Cache-miss latency (s/request) with the Predictor enabled. Reductions are largest where expert transfer dominates the critical path; on configurations where the baseline already overlaps most transfers (e.g., Qwen2-MoE on H20) the headroom is smaller, as expected.

Model	A6000	H20	Ascend 910B
DeepSeek-V2-Lite	0.044	0.001	0.062
Qwen1.5-MoE	0.038	0.038	0.069
Qwen2-MoE	0.125	1.018	–

by showing that enabling the forecaster lowers cache-miss latency across all model groups.

Tables 1–2 summarize the cache-miss latency component (seconds per request, averaged over the 410-request suite at the representative prompt-length bucket) under the baseline and the forecaster-enabled configuration; “–” denotes a configuration that is unsupported by the device’s software stack under our 20 GB cap. Values are means over three runs (per-cell standard deviation $\leq 6\%$ of the mean). The forecaster gives an order-of-magnitude or larger reduction on every supported configuration except Qwen2-MoE on H20, where the baseline already overlaps most transfers; the latency reduction there is correspondingly small but never negative. The forecaster’s robustness to long horizons is what lets the controller push S higher when bandwidth is the bottleneck without paying a misprediction tax.

5.4 Q2: Memory Management Policy

We evaluate the contribution of our memory management policy, which provides fine-grained control over expert swap operations via an LRU-style residency policy. The key objective is to prevent

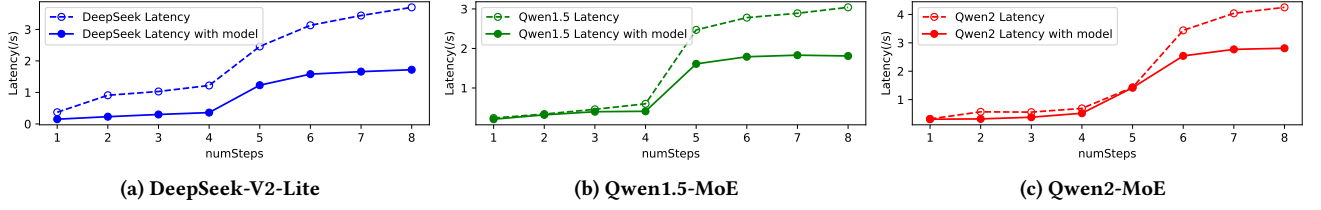


Figure 9: Cache-miss latency after enabling the pregate-based Predictor.

pathological eviction-reload cycles that amplify latency under limited GPU memory.

Fig. 10 shows that enabling the two-tier policy consistently reduces latency compared to the baseline without dedicated swap-out control. Even with the same forecasting mechanism, controlling which experts remain resident in HBM is essential for stabilizing performance under tight memory budgets. The most informative point is the latency spike that the baseline exhibits near $S \approx 4$: larger step sizes increase prefetch depth and short-term residency demand, and once this demand exceeds the memory cap, eviction-reload cycles amplify latency. The two-tier residency policy flattens this cliff into a plateau by localizing the cost of misprediction to a single DMA rather than triggering an eviction-reload cascade. This is what makes the controller’s exploration of larger S safe.

5.5 Q3: Cache-aware Routing

Finally, we evaluate cache-aware routing, which couples routing decisions with current cache residency to reduce cache misses and transfer-induced stalls. This mechanism is designed to complement both the Predictor and the memory management policy by reducing the probability that routing selects experts that are expensive to materialize under current memory pressure.

Fig. 11 shows that cache-aware routing reduces end-to-end latency across DeepSeek, Qwen1.5, and Qwen2.0 by mitigating expert-loading delays and cache misses. The relative improvement is strongest for DeepSeek and Qwen1.5, which are more sensitive to swapping-induced instability under our memory cap. Qwen2.0 benefits as well, although its built-in shared-expert design [30] already alleviates some cache inefficiencies; cache-aware routing is therefore complementary rather than redundant. Table 3 reports the cache-miss latency component (s/request, mean of three runs; same methodology as Tables 1–2) with cache-aware routing further enabled on top of the forecaster and the two-tier cache. The largest reduction is on H20, consistent with its higher compute throughput exposing relatively more transfer time when routing is cache-oblivious. Combined with the controller and forecaster, the routing layer closes the loop: prediction errors that survive the cache no longer propagate into stalls.

5.6 Runtime Overhead

We separately measure the overhead of the three layers introduced by DynaMoE to confirm that they do not displace the gains shown in Q1–Q3. The controller’s per-update cost is bounded by 1 to a few EMA updates and a comparison; we measure it at $< 50 \mu\text{s}$ on a commodity CPU across all evaluated workloads, which is at least two

Table 3: Cache-miss latency (s/request) with cache-aware routing enabled.

Model	A6000	H20	Ascend 910B
DeepSeek-V2-Lite	0.033	0.003	0.002
Qwen1.5-MoE	0.022	0.002	0.002
Qwen2-MoE	0.103	0.098	–

orders of magnitude smaller than per-layer compute on every device. The forecaster runs on the same CPU thread as the controller; its prediction at control-period granularity is sub-millisecond and never appears on the GPU critical path, so its amortized cost is dominated by the per-epoch retraining that runs between requests. The memory layer’s two-tier LRU operates on block indices and a single asynchronous DMA per eviction, with the allocator-side cost hidden by the dedicated transfer stream and not visible in the latency components reported above. Aggregated across the three layers, the wall-clock overhead introduced by DynaMoE is below the noise floor of our latency-attribution instrumentation, so the cache-miss reductions in Tables 1–3 are net of all framework costs.

6 Discussion and Limitations

Workload regimes with limited gains. DynaMoE targets the regime in which expert transfer is on (or near) the critical path. When effective transfer bandwidth is high enough that the baseline already overlaps most transfers (e.g., Qwen2-MoE on H20 in § 5), the controller converges to a small S and contributes only the $O(L_{\text{MoE}})$ controller overhead, which we measure to be under $50 \mu\text{s}$ per update on a commodity CPU. We did not observe a configuration in which DynaMoE degraded the baseline, but we do not claim universal speedups: a model whose entire expert footprint fits in HBM under the deployment cap, or a workload whose routing is so concentrated that one or two experts cover it, gains little from adaptive prefetching.

Memory-cap sensitivity. Our evaluation fixes a 20 GB HBM cap to expose expert-residency contention on every device under study. The cap was chosen to emulate consumer GPUs and multi-tenant deployments where KV-cache, activations, and other tenants compete for HBM with expert weights; it does not represent the only operating point of interest. The behavior of DynaMoE at smaller caps (10 GB) is bounded by the $S_{\min}$ floor of the controller, which prevents thrashing at the cost of losing some overlap; at larger caps the controller converges to higher S and gains diminish as more experts become persistently resident. Characterizing this curve

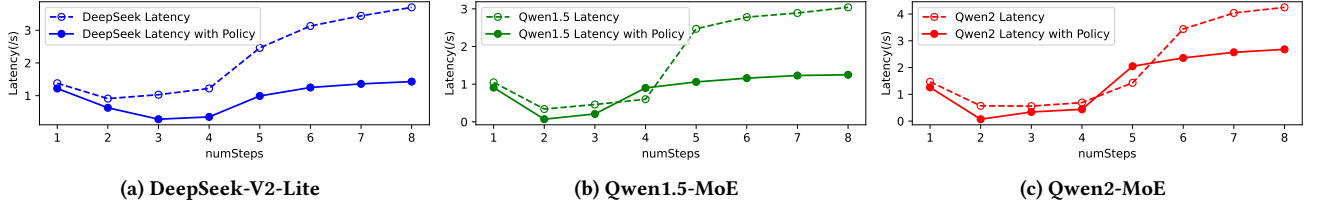


Figure 10: Latency under cross-layer prediction with the proposed memory management policy.

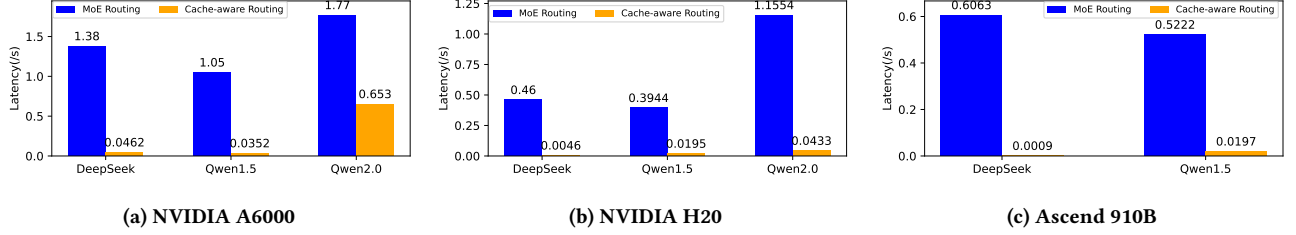


Figure 11: End-to-end latency comparison with and without cache-aware routing.

quantitatively is left to future work; we expect the qualitative trend to mirror Fig. 2, since the underlying overlap-pollution tradeoff is the same.

Multi-stream and multi-tenant deployment. DynaMoE targets a single-stream serving setup with continuous batching. Co-locating multiple model instances on a single GPU introduces inter-instance contention on the transfer stream and on HBM, which the current controller does not explicitly model: each instance optimizes its own S , and contention manifests in the shared bandwidth estimate C_s . We expect the per-instance controller to remain stable (because (C2) bounds per-step change to ± 1), but the joint optimum may not match the union of local optima. Lifting the controller to a coordinated multi-stream setting—potentially by sharing C_s and $\bar{m}_t$ across instances—is a natural extension.

Predictor cold start and online retraining. The forecasting layer relies on a CPU-side random forest trained offline from logged tuples. On a previously unseen model or substantially shifted workload, the forecaster falls back to the model’s own pre-gate top- k , which we measured matches the fixed-horizon baseline. The system then degrades gracefully rather than catastrophically. Online retraining (incremental forest updates as new (t, ℓ, S, a) tuples arrive) is straightforward to add and does not interact with the controller’s stability proof, but we have not characterized its convergence empirically.

Threats to generality. Our experiments cover three MoE models from the DeepSeek and Qwen families and three accelerators spanning a $4\times$ effective-bandwidth range. We did not evaluate Mixtral-class models (which use a different expert sizing) or larger DeepSeek-V3-class models; we expect the same mechanism to apply, but the optimal hyperparameters (λ, θ_h) may shift. We also do not evaluate disaggregated or pipeline-parallel deployments, where prefetch coordination crosses device boundaries.

7 Related Work

7.1 MoE Inference Systems

Early sparse-activation designs such as the Switch Transformer [10] demonstrated that routing each token to a small subset of experts substantially reduces computation while preserving model capacity. Subsequent serving systems strengthened this direction by addressing communication and scheduling costs at large scale [16, 21, 33, 35, 38]. ExFlow [32] leverages inter-layer expert affinity to colocate frequently co-activated experts, while TA-MoE [5] uses device topology to reduce cross-node transfers. These approaches reduce communication but assume that expert availability can be largely predicted offline.

A separate line of work targets memory pressure on commodity or memory-constrained GPUs. ProMoE [23], ExpertFlow [12], and Hobbit [25] all employ cross-layer prefetching with a fixed forecasting horizon. ExpertFlow integrates token-aware expert prediction with proactive caching; Hobbit relies on mixed-precision offloading and multi-level caches; MoE-Lightning [4] introduces hierarchical memory management to rebalance expert residency mid-execution. Other systems prioritize experts already in fast memory [2] or inject null experts to smooth routing dynamics [34]. DynaMoE differs from all of the above by treating the prefetch horizon as a runtime control variable rather than an offline-tuned hyperparameter, by closing a feedback loop between stall/eviction signals and the horizon decision, and by letting cache-aware token routing absorb residual misprediction inside resident-expert compute.

7.2 Routing and Load Balancing

Routing quality is a central determinant of MoE efficiency [13, 36]. Expert Choice [37] provides load balancing by letting experts self-select tokens; AdaMoE [34] dynamically adjusts the number of active experts per token; ExFlow [12] maintains routing consistency across layers to reduce communication; MoDSE [24] assigns

heterogeneous expert sizes to match input complexity. These techniques shape *which* experts are activated. DynaMoE is orthogonal: rather than designing new routing objectives or expert topologies, we treat the cross-layer prediction interval as a runtime control variable, leaving routing semantics unchanged. The two directions are complementary, and DynaMoE can be deployed in front of any of the routing schemes above.

8 Conclusion

In this paper we present DynaMoE, a serving runtime designed for memory-constrained Mixture-of-Experts inference under expert-residency contention. DynaMoE realizes the idea of treating the cross-layer prefetch horizon S as a *runtime control variable*: a bounded ± 1 update with hysteresis on EMA-smoothed stall and eviction signals picks a different S_t at each control period. This reframing breaks the brittle dependence of fixed-horizon prefetching on hardware and workload, recovering the per-configuration optimum that no single offline S can match. A CPU-side delta-adjusted forecaster and a residency-aware two-tier expert cache with cache-aware token routing make this control loop safe by bounding the cost of every prediction error to a single DMA. Our evaluation across three accelerators and three MoE models demonstrates that DynaMoE reduces cache-miss latency by an order of magnitude on most configurations while never degrading the ones where the baseline is already near-optimal.

References

- [1] Anonymous Contributors. 2023. ShareGPT Vicuna Unfiltered. https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered. Hugging Face dataset.
- [2] Abhimanyu Rajeshkumar Bambhaniya, Sashankh Chengavalli Kumar, and Tushar Krishna. 2024. MoE-ERAS: Expert Residency Aware Selection. In *Machine Learning for Computer Architecture and Systems 2024*.
- [3] Xiao Bi, Deli Chen, Quanting Chen, Shanhua Chen, Damai Dai, Chengqi Deng, Honghui Ding, Kai Dong, Qiusi Du, Zhe Fu, et al. 2024. Deepseek llm: Scaling open-source language models with longtermism. *arXiv preprint arXiv:2401.02954* (2024).
- [4] Shiyi Cao et al. 2024. MoE-Lightning: High-Throughput MoE Inference on Memory-constrained GPUs. *arXiv preprint arXiv:2411.11217* (2024).
- [5] Chang Chen, Min Li, Zhihua Wu, Dianhai Yu, and Chao Yang. 2022. Ta-moe: Topology-aware large scale mixture-of-expert training. *Advances in Neural Information Processing Systems* 35 (2022), 22173–22186.
- [6] Zixiang Chen, Yihe Deng, Yue Wu, Quanquan Gu, and Yuanzhi Li. 2022. Towards understanding the mixture-of-experts layer in deep learning. *Advances in neural information processing systems* 35 (2022), 23049–23062.
- [7] DeepSeek AI. 2024. DeepSeek-Coder-V2-Lite-Base. <https://huggingface.co/deepseek-ai/DeepSeek-Coder-V2-Lite-Base>. Hugging Face model card.
- [8] DeepSeek-AI et al. 2024. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. *arXiv:2405.04434* [cs.CL] <https://arxiv.org/abs/2405.04434>
- [9] Artyom Eliseev and Denis Mazur. 2023. Fast inference of mixture-of-experts language models with offloading. *arXiv preprint arXiv:2312.17238* (2023).
- [10] William Fedus, Barret Zoph, and Noam Shazeer. 2021. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. *arXiv:2101.03961* [cs.LG]
- [11] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research* 23, 120 (2022), 1–39.
- [12] Xin He, Shunkang Zhang, Yuxin Wang, Haiyan Yin, Zihao Zeng, Shaohuai Shi, Zhenheng Tang, Xiaowen Chu, Ivor Tsang, and Ong Yew Soon. 2024. Expertflow: Optimized expert activation and token allocation for efficient mixture-of-experts inference. *arXiv preprint arXiv:2410.17954* (2024).
- [13] Hukai Huang, Shenghui Lu, Yahui Shan, He Qu, Fengrun Zhang, Wenhao Guan, Qingyang Hong, and Lin Li. 2025. Dynamic Language Group-based MoE: Enhancing Code-Switching Speech Recognition with Hierarchical Routing. In *ICASSP 2025-2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 1–5.
- [14] Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. 2024. Mixtral of experts. *arXiv preprint arXiv:2401.04088* (2024).
- [15] Dario Lepikhin, Yiwen Lee, Yi Xu, Mingxing Chen, Oriol Vinyals, Yiming Huang, and Barret Zoph. 2020. GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding. *arXiv:2006.16668* [cs.LG]
- [16] Yunxin Li, Shenyuan Jiang, Baotian Hu, Longyue Wang, Wanqi Zhong, Wenhan Luo, Lin Ma, and Min Zhang. 2025. Uni-moe: Scaling unified multimodal llms with mixture of experts. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2025).
- [17] Saeed Masoudnia and Reza Ebrahimpour. 2014. Mixture of experts: a literature survey. *Artificial Intelligence Review* 42, 2 (2014), 275–293.
- [18] NVIDIA Corporation. 2024. NVIDIA RTX A6000 Datasheet. <https://resources.nvidia.com/en-us-briefcase-for-datasheets/proviz-print-nvidia-1?ncid=no-ncid>. Accessed: 2025-06-14.
- [19] Qwen Team. 2024. Qwen1.5-MoE-A2.7B. <https://huggingface.co/Qwen/Qwen1.5-MoE-A2.7B>. Hugging Face model card.
- [20] Qwen Team. 2024. Qwen2-57B-A14B-Instruct-GPTQ-Int4. <https://huggingface.co/Qwen/Qwen2-57B-A14B-Instruct-GPTQ-Int4>. Hugging Face model card.
- [21] Carlos Riquelme et al. 2021. Scaling vision with sparse mixture of experts. *Advances in Neural Information Processing Systems* 34 (2021), 8583–8595.
- [22] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc V. Le, Geoffrey Hinton, and Jeffrey Dean. 2017. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. *arXiv:1701.06538* [cs.LG]
- [23] Xiaoni Song, Zihang Zhong, and Rong Chen. 2024. ProMoE: Fast MoE-based LLM Serving using Proactive Caching. *arXiv preprint arXiv:2410.22134* (2024).
- [24] Manxi Sun, Wei Liu, Jian Luan, Pengzhi Gao, and Bin Wang. 2024. Mixture of Diverse Size Experts. *arXiv preprint arXiv:2409.12210* (2024).
- [25] Peng Tang, Jiacheng Liu, Xiaofeng Hou, Yifei Pu, Jing Wang, Pheng-Ann Heng, Chao Li, and Minyi Guo. 2024. Hobbit: A mixed precision expert offloading system for fast moe inference. *arXiv preprint arXiv:2411.01433* (2024).
- [26] Qwen Team. 2024. Qwen1.5-MoE: Matching 7B Model Performance with 1/3 Activated Parameters". <https://qwenlm.github.io/blog/qwen-moe/>
- [27] Qwen Team et al. 2024. Qwen2 technical report. *arXiv preprint arXiv:2407.10671* 2, 3 (2024).
- [28] An Wang, Xingwu Sun, Ruobing Xie, Shuaipeng Li, Jiaqi Zhu, Zhen Yang, Pinxue Zhao, Weidong Han, Zhanhui Kang, Di Wang, et al. 2025. Hmoe: Heterogeneous mixture of experts for language modeling. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 21954–21968.
- [29] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations*. 38–45.
- [30] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. 2024. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115* (2024).
- [31] Yuanhang Yang, Shiyi Qi, Wenchao Gu, Chaozheng Wang, Cuiyun Gao, and Zenglin Xu. 2024. XMoE: Sparse Models with Fine-grained and Adaptive Expert Selection. In *Findings of the Association for Computational Linguistics ACL 2024*. 11664–11674.
- [32] Jinghan Yao, Quentin Anthony, Aamir Shafi, Hari Subramoni, and Dhableswar K DK Panda. 2024. Exploiting inter-layer expert affinity for accelerating mixture-of-experts model inference. In *2024 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 915–925.
- [33] Enda Yu, Zhaoning Zhang, Dezun Dong, Yongwei Wu, and Xiangke Liao. 2025. PreScope: Unleashing the Power of Prefetching for Resource-Constrained MoE Inference. *arXiv:2509.23638* [cs.LG] <https://arxiv.org/abs/2509.23638>
- [34] Zihao Zeng, Yibo Miao, Hongcheng Gao, Hao Zhang, and Zhijie Deng. 2024. AdaMoE: Token-Adaptive Routing with Null Experts for Mixture-of-Experts Language Models. *arXiv preprint arXiv:2406.13233* (2024).
- [35] Yuning Zhang, Grant Pinkert, Nan Yang, Yanli Li, and Dong Yuan. 2025. DuoServe-MoE: Dual-Phase Expert Prefetch and Cache Scheduling for Efficient MoE LLM Inference. *arXiv:2509.07379* [cs.DC] <https://arxiv.org/abs/2509.07379>
- [36] Hao Zhou, Zhijun Wang, Shujian Huang, Xin Huang, Xue Han, Junlan Feng, Chao Deng, Weihua Luo, and Jiajun Chen. 2025. Moe-lpr: Multilingual extension of large language models through mixture-of-experts with language priors routing. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 26092–26100.
- [37] Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew M Dai, Quoc V Le, James Laudon, et al. 2022. Mixture-of-experts with expert choice routing. *Advances in Neural Information Processing Systems* 35 (2022), 7103–7114.
- [38] Jiapeng Zhu, Ceyuan Yang, Kecheng Zheng, Yinghao Xu, Zifan Shi, Yifei Zhang, Qifeng Chen, and Yujun Shen. 2025. Exploring sparse MoE in GANs for text-conditioned image synthesis. In *Proceedings of the Computer Vision and Pattern Recognition Conference*. 18411–18423.